IEEE
ASU STUDENT BRANCH

Computer Arithmetic

Session 3 - Multipliers

By: Abdallah Zein

IEEE
ASU STUDENT BRANCH

Multioperand Addition

Motivation

We previously covered several ways to speed up adding 2 operands,

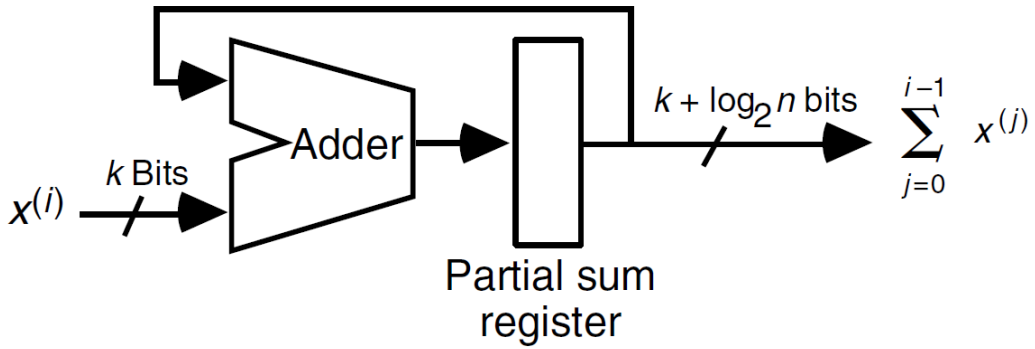
What if we want to add 3 operands, 4,…, N operands?

Using 2 operand adders we can design find:

- Serial Solution
- Some Sort of tree solution

Serial Implementation

Using a single 2-operands adder, one operand can be applied to one adder input, while the other input is fed back from an accumulation (partial sum) register, needing **n cycles** to add **n operands.**



$$i = 0, 1, \cdots, n-1$$

Why the output is $k + \log_2(n)$ bits wide?

Multi-op Addition

Serial Implementation

Why the output is $k + \log_2(n)$ bits wide?

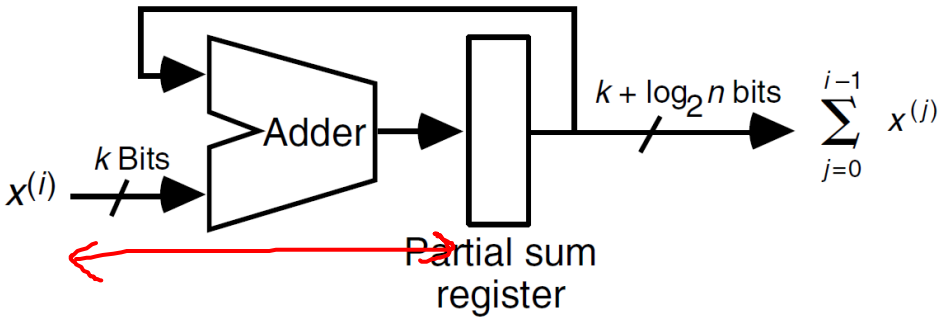
If the operands are k bits wide, then the maximum operand value is $2^k - 1$, this makes the maximum output value is $n(2^k - 1)$.

This can be represented in $\log_2\left(n(2^k - 1) + 1\right)$ bits

$$\approx \log_2\left(n(2^k - 1)\right) = \log_2(2^k - 1) + \log_2(n) \approx k + \log_2(n)$$

Assuming we used a logarithmic-fast adder:

$$T_{\text{serial}-\text{multi}-\text{add}} = O(n \log(k + \log n))$$
$$= O(n \log\ k + n \log\ \log n)$$



$$i = 0, 1, \cdots, n-1$$

Multi-op Addition

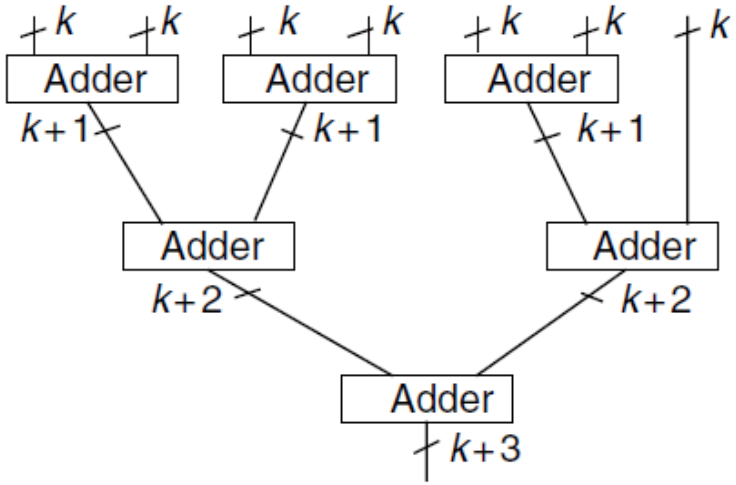
Tree Implementation

For higher speeds, a tree of adders may be used, let's take a binary tree of 2 operand adders for example:

For n operands, we need n-1 adders, which is costly if built with logarithmic-fast adders.

Strangely as it may sounds, using the slow ripple carry adders may be the best choice for this design, not just for cost, but also for **SPEED!**

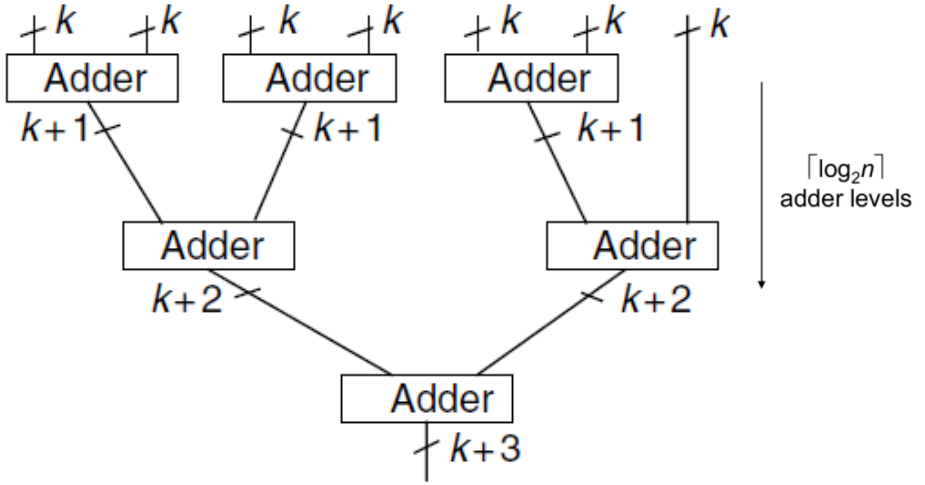
What is the total latency in case of O(log(n)) adders?



Tree Implementation

What is the total latency in case of O(log(k)) adders?

$$T_{\text{tree-fast-multi-add}} = \mathrm{O}(\log\ k + \log(k+1) + \cdots + \log(k + \lceil \log_2 n \rceil - 1))$$
$$= \mathrm{O}(\log\ n\ \log\ k + \log\ n \log\ \log\ n)$$

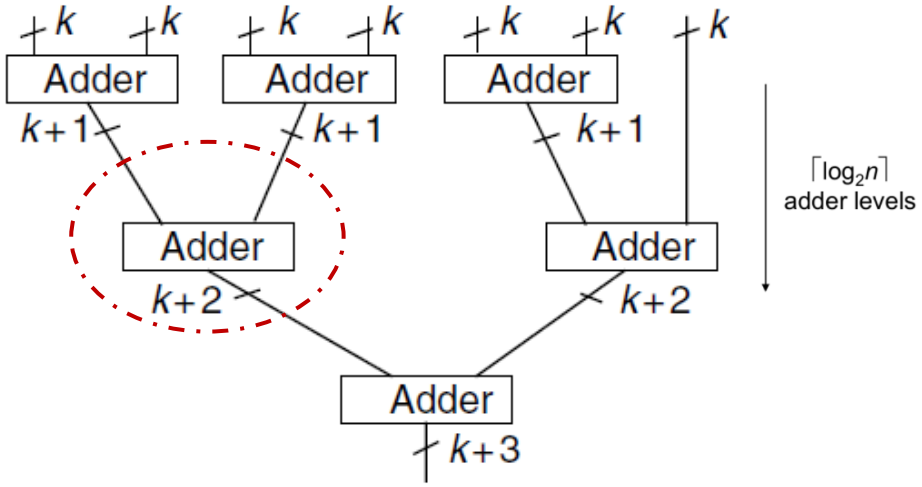


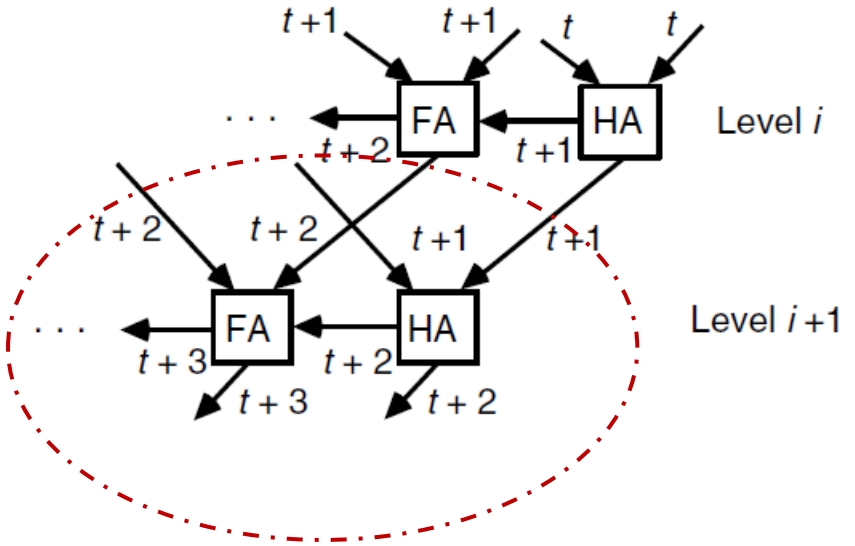
On the other side, when using ripple carry adders, we will find that the latency is given by:

$$T_{\text{tree-ripple-multi-add}} = \mathrm{O}(k + \log\ n)$$

How?

Tree Implementation





$$T_{\text{tree-ripple-multi-add}} = O(k + \log\ n)$$

Can we do better?

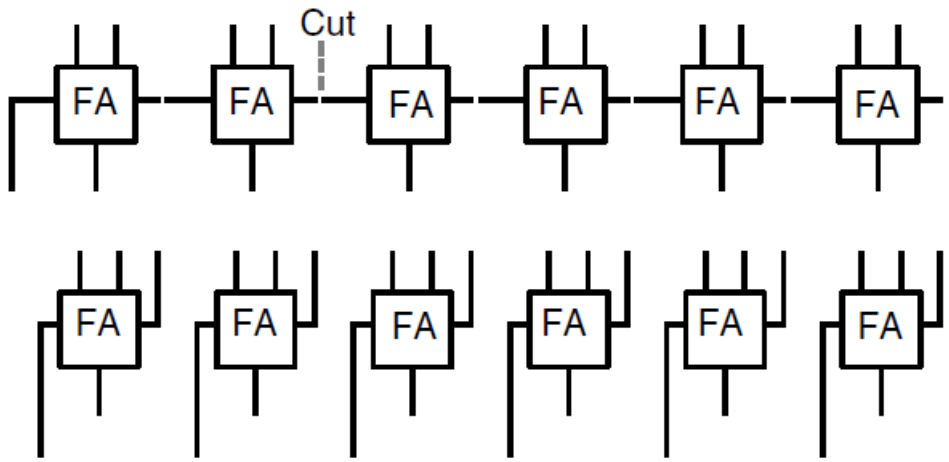
Multi-op Addition

Tree Implementation

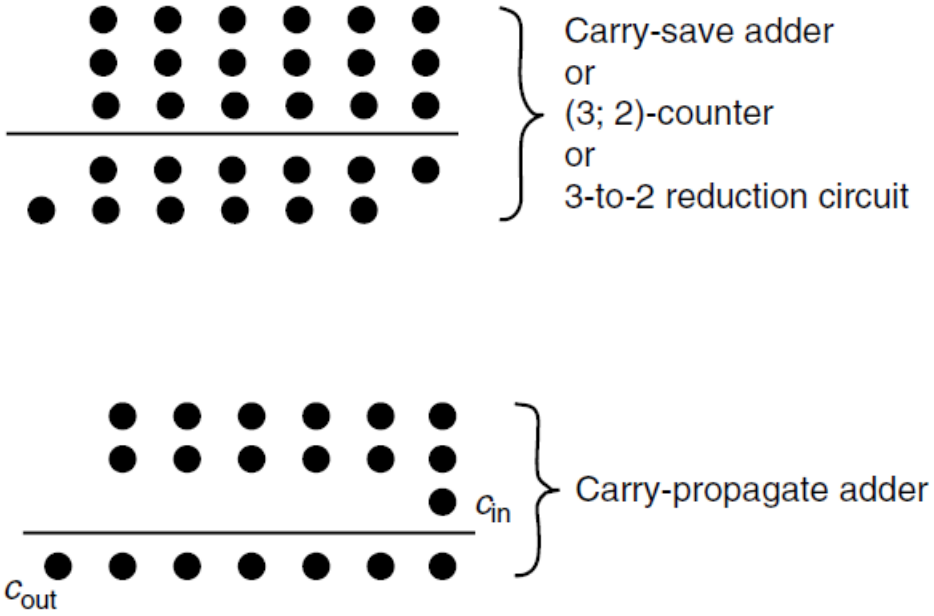
For n operands each of k bits wide, we have a total of kn bits to process, the absolute minimum latency we can achieve is O(log(kn)) = **O(log k + log n)**

This could be achieved using **Carry Save Adders**

We can see a row of FAs as a mechanism that reduces 3 numbers to 2, instead of two numbers to their sum…what does this mean?



CSA vs CPA



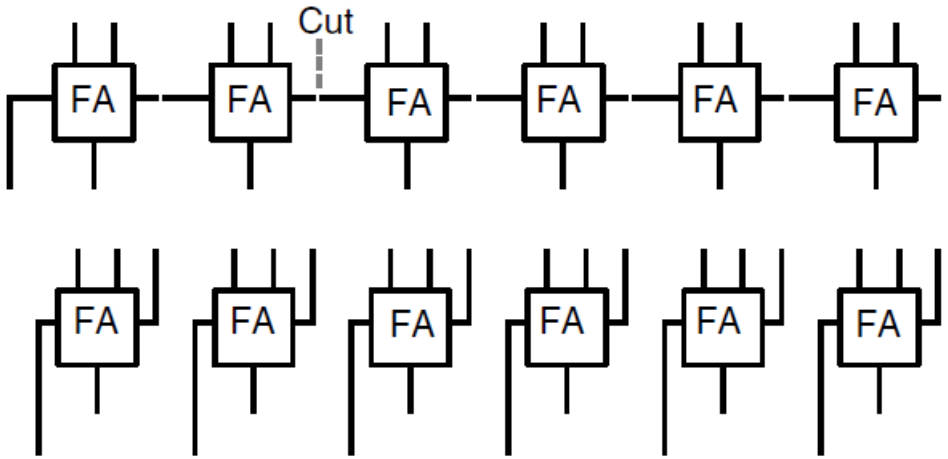

```
  0 0 1 0 1 1
    1 0 1 0 1 1
  ---------------
    0 1 1 1 1 1
            0
  ---------------
  0 0 1 0 1 1
    1 1 0 1 0 0
  ---------------
  1 0 0 1 0 1 0
```

Multi-op Addition

CSA vs CPA

The Carry Save Adder **saves** the carry bits generated during addition **as a separate number** rather than **propagating** them immediately to the next stage.

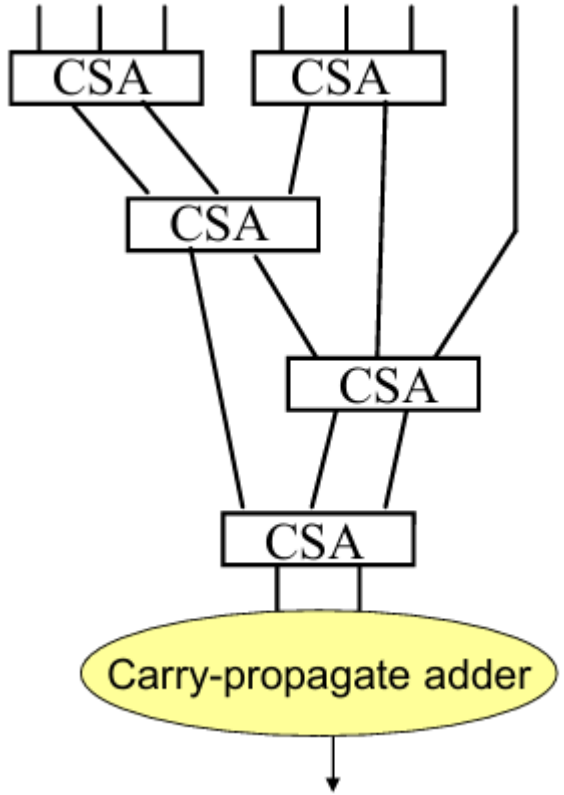
By storing these carry bits and adding them in a final, separate CPA stage, the CSA avoids the long delays of carry propagation, enabling high-speed parallel addition of multiple operands.



Carry-Save Adders

Multioperand Addition Using CSAs

- Tree of carry-save adders reducing seven numbers to two. **How many CSAs are needed?**
- The needed CSAs are of various widths, but generally the widths are close to k bits; the CPA is of width at most k +log2n



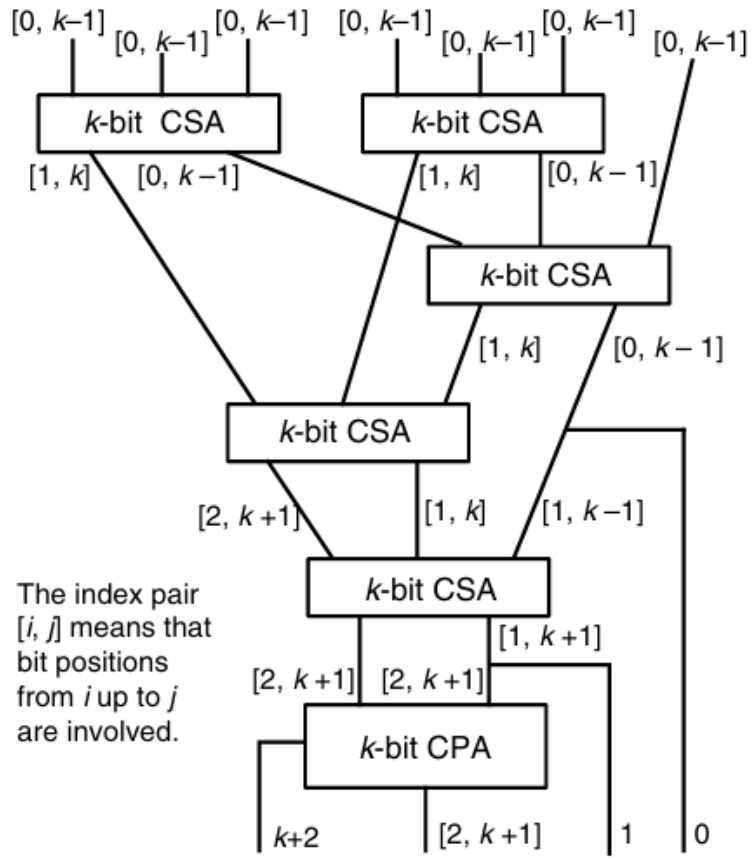
$$T_{\text{carry-save-multi-add}} = O(\text{tree height} + T_{CPA})$$
$$= O(\log n + \log k)$$

$$C_{\text{carry-save-multi-add}} = (n-2)C_{CSA} + C_{CPA}$$

Multioperand Additiion

Carry-Save Adders

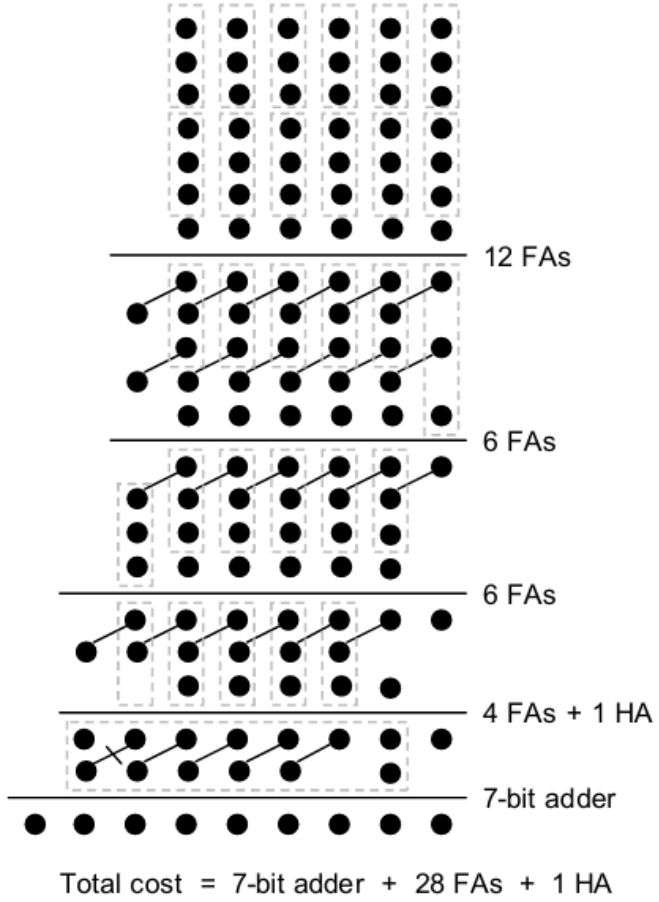
Width of Adders in a CSA Tree



- Adding Seven k-bit numbers and the CSA/CPA Widths required
- Due to the gradual retirement (dropping out) of some of the result bits, CSA widths do not vary much as we go down the tree levels

Carry-Save Adders

Example Reduction by a CSA Tree



Total cost = 7-bit adder + 28 FAs + 1 HA

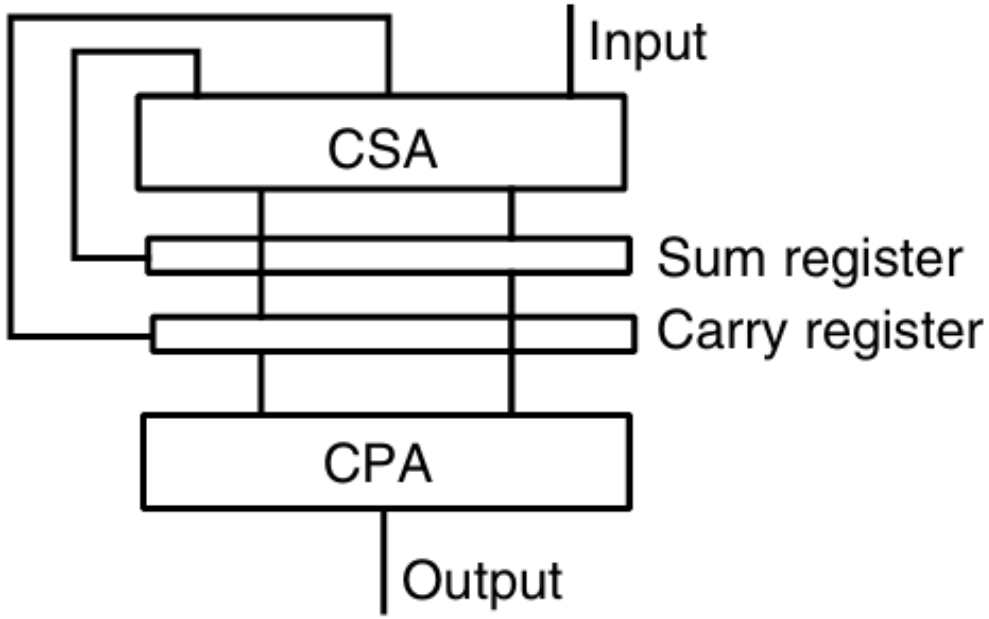
8	7	6	5	4	3	2	1	0	Bit position
			7	7	7	7	7	7	6×2 = 12 FAs
		2	5	5	5	5	5	3	6 FAs
		3	4	4	4	4	4	1	6 FAs
	1	2	3	3	3	3	2	1	4 FAs + 1 HA
	2	2	2	2	2	1	2	1	7-bit adder
--Carry-propagate adder--									
1	1	1	1	1	1	1	1	1	

Multioperand Additiion

Carry-Save Adders

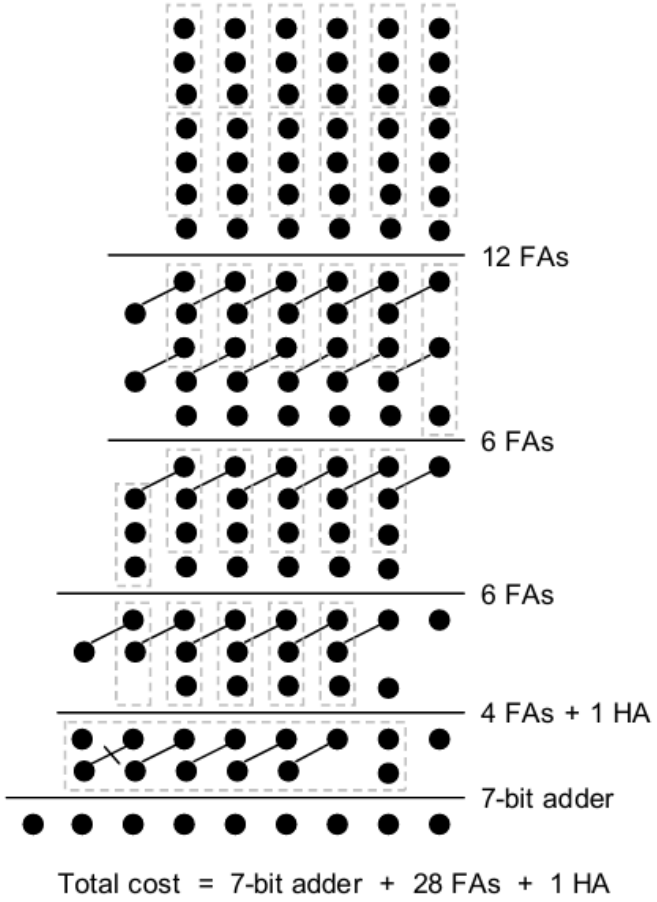
Serial Carry-Save Addition by means of a single CSA

- better implementation for the serial addition technique (smaller cycle delay single FA)



Wallace Trees

Reducing the Number of Operands as Early as possible



Total cost = 7-bit adder + 28 FAs + 1 HA

$$r_{j+1} = 2 * floor\left(\frac{r_j}{3}\right) + r_j \bmod(3)$$

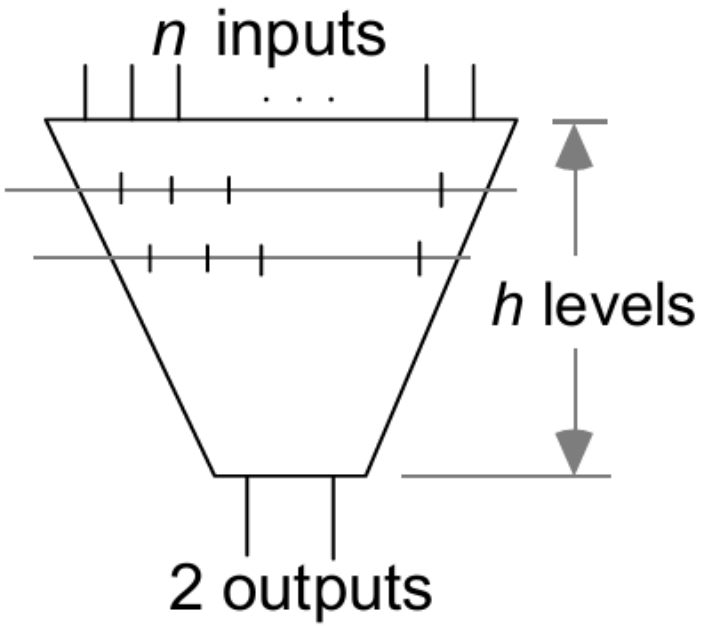
3 Comes from Full Adder is 3:2 Compressor

here in this example: $r_0 = 6 \rightarrow r_1 = 4 \rightarrow r_2 = 3 \rightarrow r_3 = 2$

CSA Reduction Tree

Maximum Number of Operands per Level

- $d_{k+1} = floor\left(\frac{3}{2} \times d_k\right)$
- $d_0 = 2$
- $d_0 = 2 \rightarrow d_1 = 3 \rightarrow d_2 = 4 \ldots$



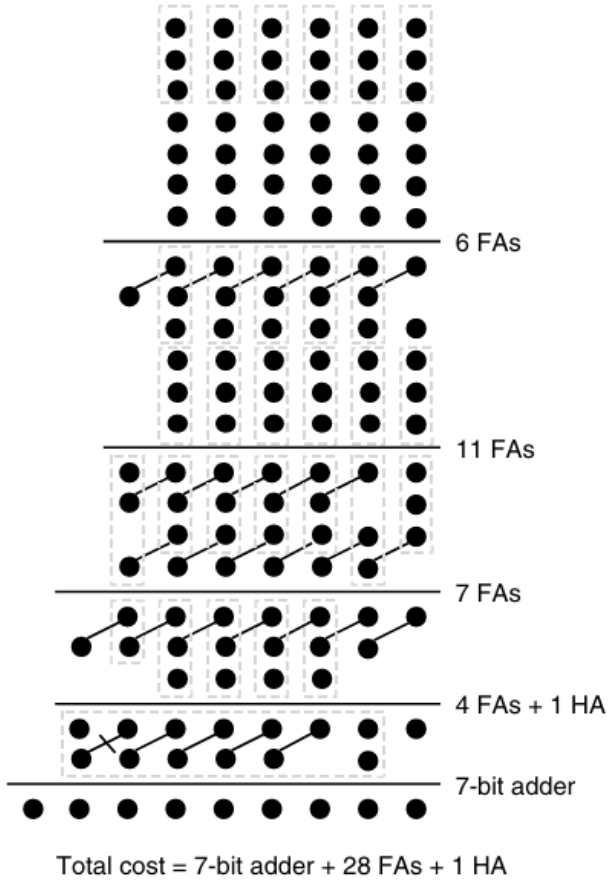
h	n(h)	h	n(h)	h	n(h)
0	2	7	28	14	474
1	3	8	42	15	711
2	4	9	63	16	1066
3	6	10	94	17	1599
4	9	11	141	18	2398
5	13	12	211	19	3597
6	19	13	316	20	5395

n(*h*): Maximum number of inputs for *h* levels

Dadda Trees

Column Height Recurrence per Level

- $d_{k+1} = floor\left(\frac{3}{2} \times d_k\right)$
- Initial d < number of operands
- EX: Number of operands = 7
- Start from the largest valid 2 and reduce to 2
- d=6 → d=4 → d=3 → d=2 Finish
- Dadda tree uses fewer reduction stages than Wallace
- **Reductions are postponed as much as possible without increasing delay**
- *This minimizes the number of adders while preserving optimal delay.*



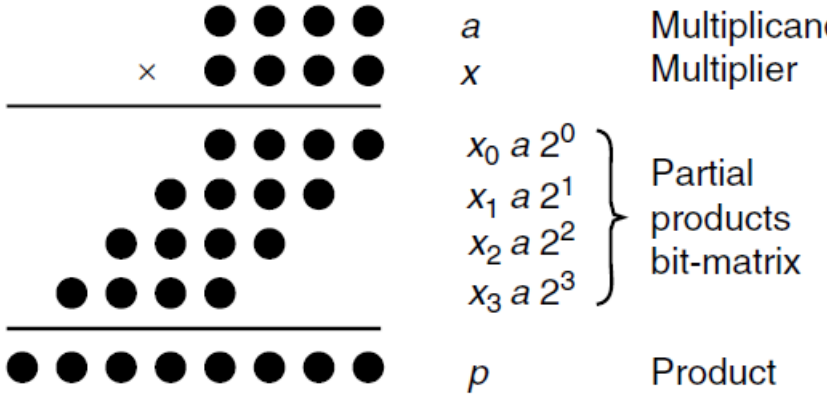
IEEE
ASU STUDENT BRANCH

Multiplication

Multiplication

The following notation is used in our discussion of multiplication algorithms:

a	Multiplicand	$a_{k-1}a_{k-2}\cdots a_1a_0$
x	Multiplier	$x_{k-1}x_{k-2}\cdots x_1x_0$
p	Product $(a \times x)$	$p_{2k-1}p_{2k-2}\cdots p_1p_0$



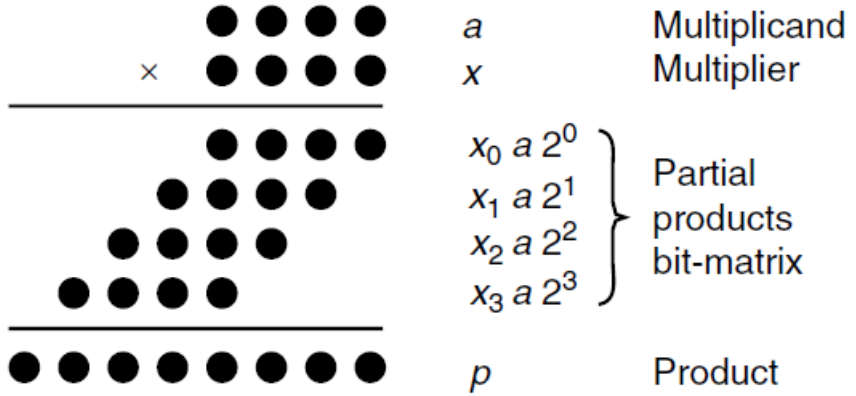
In multiplying a multiplicand a by a k-digit multiplier x, the k partial products $x_i a$ must be formed and then added. We can see that it can be simplified into a series of shift and add operations.

Basic Multiplication

Multiplication

Each of the following four rows of dots corresponds to the product of the multiplicand a and 1 bit of the multiplier x, since x_i is just either 0 or 1, then the partial product p_i is either 0 or a. therefore the problem of binary multiplication reduces to adding a set of numbers, each of which 0 or a shifted version of a.

Multioperand Addition!



Basic Multiplication

Sequential Multiplication

The use of a shift add algorithm, gives the rise of two versions of the algorithm, the right shift and the left shift, depending on whether the partial product terms are processed from top to bottom or the other way around.

$$p^{(j+1)} = (p^{(j)} + x_j a\ 2^k)2^{-1} \quad \text{with} \quad p^{(0)} = 0 \text{ and } p^{(k)} = p$$

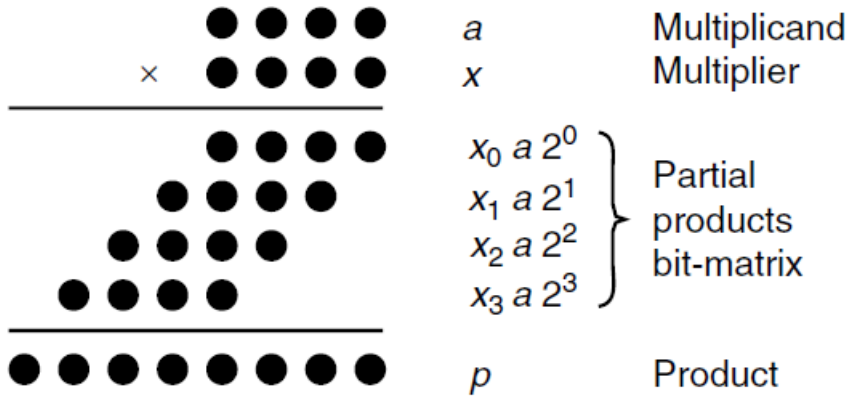
$$\vdash\!\!\text{add}\!\!\dashv$$

$$\vdash\!\!\text{shift right}\!\!\dashv$$

$$p^{(j+1)} = 2p^{(j)} + x_{k-j-1}a \quad \text{with} \quad p^{(0)} = 0 \text{ and } p^{(k)} = p$$

$$\left|\begin{matrix}\text{shift}\\ \text{left}\end{matrix}\right|$$

$$\vdash\!\!\text{add}\!\!\dashv$$



Basic Multiplication

Sequential Multiplication

Example:

Multiplying $a=(10)_{10}$ by $x=(11)_{10}$

The two algorithms are quite similar, how ever additions in the left shift are 2k bits wide, while the right shift is only k bits, that's why right shift is preferable.

(a) Right-shift algorithm

a		1	0	1	0		
x		1	0	1	1		
$p^{(0)}$		0	0	0	0		
$+x_0a$		1	0	1	0		
$2p^{(1)}$	0	1	0	1	0		
$p^{(1)}$		0	1	0	1	0	
$+x_1a$		1	0	1	0		
$2p^{(2)}$	0	1	1	1	1	0	
$p^{(2)}$		0	1	1	1	1	0
$+x_2a$		0	0	0	0		
$2p^{(3)}$	0	0	1	1	1	1	0
$p^{(3)}$		0	0	1	1	1	1	0
$+x_3a$		1	0	1	0		
$2p^{(4)}$	0	1	1	0	1	1	1	0
$p^{(4)}$		0	1	1	0	1	1	1	0

(b) Left-shift algorithm

a				1	0	1	0
x				1	0	1	1
$p^{(0)}$				0	0	0	0
$2p^{(0)}$			0	0	0	0	0
$+x_3a$				1	0	1	0
$p^{(1)}$			0	1	0	1	0
$2p^{(1)}$		0	1	0	1	0	0
$+x_2a$				0	0	0	0
$p^{(2)}$			0	1	0	1	0	0
$2p^{(2)}$		0	1	0	1	0	0	0
$+x_1a$				1	0	1	0
$p^{(3)}$		0	1	1	0	0	1	0
$2p^{(3)}$	0	1	1	0	0	1	0	0
$+x_0a$				1	0	1	0
$p^{(4)}$	0	1	1	0	1	1	1	0

Basic Hardware MUL

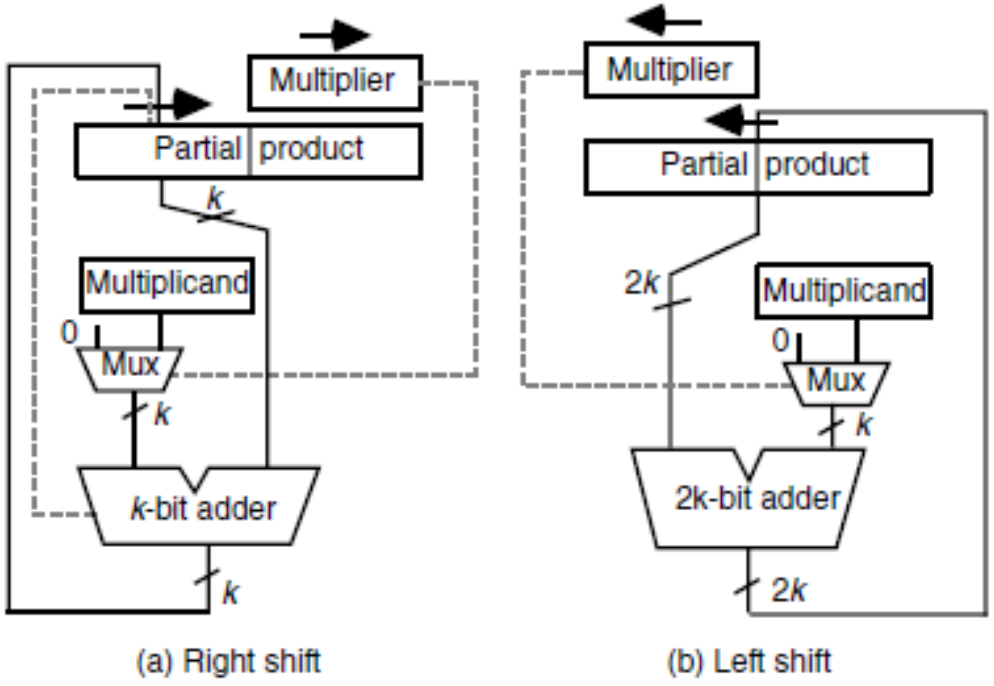
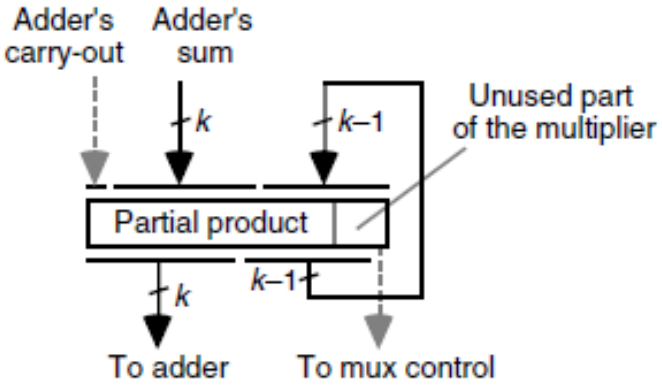


Figure 9.5 Combining the loading and shifting of the double-width register holding the partial product and the partially used multiplier.



The control portion of the multiplier, which is not shown in the Fig, it consists of a counter to keep track of the number of iterations and a simple circuit to effect initialization and detect termination.

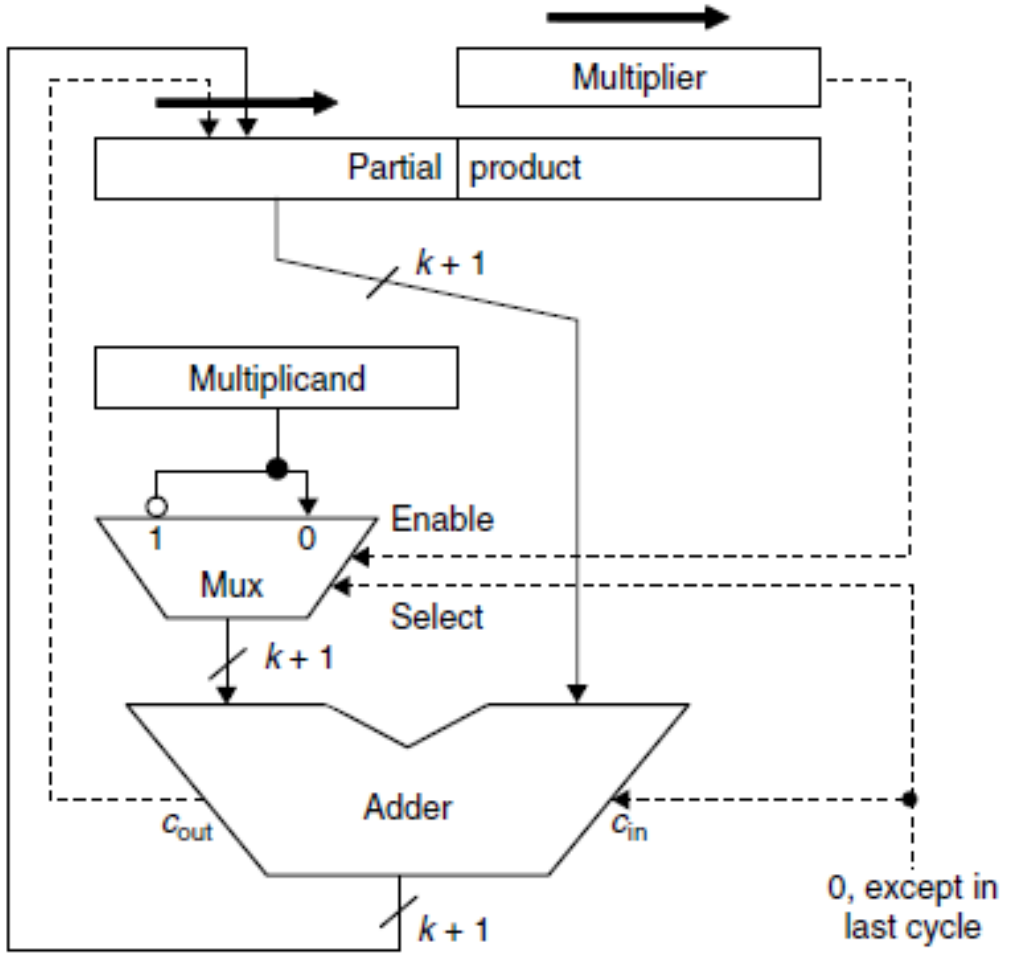
Signed Multiplication

a		1	0	1	1	0				
x		0	1	0	1	1				
$p^{(0)}$		0	0	0	0	0				
$+x_0 a$		1	0	1	1	0				
$2p^{(1)}$	1	1	0	1	1	0				
$p^{(1)}$		1	1	0	1	1	0			
$+x_1 a$		1	0	1	1	0				
$2p^{(2)}$	1	1	0	0	0	1	0			
$p^{(2)}$		1	1	0	0	0	1	0		
$+x_2 a$		0	0	0	0	0				
$2p^{(3)}$	1	1	1	0	0	0	1	0		
$p^{(3)}$		1	1	1	0	0	0	1	0	
$+x_3 a$		1	0	1	1	0				
$2p^{(4)}$	1	1	0	0	1	0	0	1	0	
$p^{(4)}$		1	1	0	0	1	0	0	1	0
$+x_4 a$		0	0	0	0	0				
$2p^{(5)}$	1	1	1	0	0	1	0	0	1	0
$p^{(5)}$		1	1	1	0	0	1	0	0	1	0

a		1	0	1	1	0				
x		1	0	1	0	1				
$p^{(0)}$		0	0	0	0	0				
$+x_0 a$		1	0	1	1	0				
$2p^{(1)}$	1	1	0	1	1	0				
$p^{(1)}$		1	1	0	1	1	0			
$+x_1 a$		0	0	0	0	0				
$2p^{(2)}$	1	1	1	0	1	1	0			
$p^{(2)}$		1	1	1	0	1	1	0		
$+x_2 a$		1	0	1	1	0				
$2p^{(3)}$	1	1	0	0	1	1	1	0		
$p^{(3)}$		1	1	0	0	1	1	1	0	
$+x_3 a$		0	0	0	0	0				
$2p^{(4)}$	1	1	1	0	0	1	1	1	0	
$p^{(4)}$		1	1	1	0	0	1	1	1	0
$+(-x_4 a)$		0	1	0	1	0				
$2p^{(5)}$	0	0	0	1	1	0	1	1	1	0
$p^{(5)}$		0	0	0	1	1	0	1	1	1	0

Basic Multiplication

Signed Multiplication



Basic Multiplication

IEEE
ASU STUDENT BRANCH

Thank You!